From Block Data to Slides

A reproducible pipeline for spatial energy analysis
with QGIS, Python, LaTeX and TikZ

Jan Suchanek

November 2026

Abstract

This document describes a pipeline that turns public, block-level energy data into maps, charts and in-text figures for reports and slide decks. It was developed in 2024–2025 for an internal analysis of energy use at eleven science and technology sites in Berlin. That study is not public; this document therefore uses *synthetic data* for five fictitious sites. The document is its own minimal working example: every chart and every number in the text is computed from the data files when the document is compiled.

Contents

1 Why a pipeline? **1**

2 Architecture **1**

3 Stage 1 and 2: data and spatial join **2**

3.1 Sources 2

3.2 Selection rule: within or intersecting? 2

4 Stage 3: preparation **3**

4.1 Extrapolating withheld values 3

4.2 Emissions 3

4.3 Build-out factor 4

4.4 Output: data and dictionary 4

5 The data dictionary **4**

6 Stage 4: drawing with `zkopipeline` **5**

6.1 Circle map 5

6.2 Projection overlay 5

6.3 Energy mix 6
6.4 Bar chart 7

7 Numbers in the text **7**

7.1 An example paragraph 8
7.2 Share, growth, ratio 8
7.3 Rounding and locale 8

8 Lessons learned **9**

9 Building **9**

A Macro reference **10**

1 Why a pipeline?

The original question was simple: how much energy do the sites consume today, from which sources, and how much will they need once they are fully built? Answering it meant combining several public data sets, each at a different spatial level, for eleven very different sites, and presenting the result repeatedly as data were corrected and extended.

Doing this by hand (copying numbers from spreadsheets into slides) fails in a predictable way: the numbers in the text drift away from the numbers in the charts. The pipeline avoids this by keeping a single source of truth. Every figure that appears in the output is *computed* from one data file at compile time. When the data change, recompiling updates every chart and every sentence.

Section 8 lists what went wrong in the first version, and how this version prevents it. Those lessons are the main reason this document exists.

2 Architecture

Stage	Tool	Responsibility
1 Raw data	public portals	Block-level consumption, buildings, businesses, site boundaries as GeoJSON
2 Spatial join	QGIS	Assign blocks to sites; export one row per block
3 Preparation	Python	Aggregate per site, extrapolate missing values, derive indicators; write the data dictionary
4 Presentation	LaTeX, TikZ, datatool	Draw maps and charts; compute every number used in the text
5 Output	PDF	Report, slide deck, one poster per site

Table 1: Stages and their responsibilities.

Two rules hold the design together:

1. **One direction.** Data flow from left to right only. Nothing is typed into a later stage that

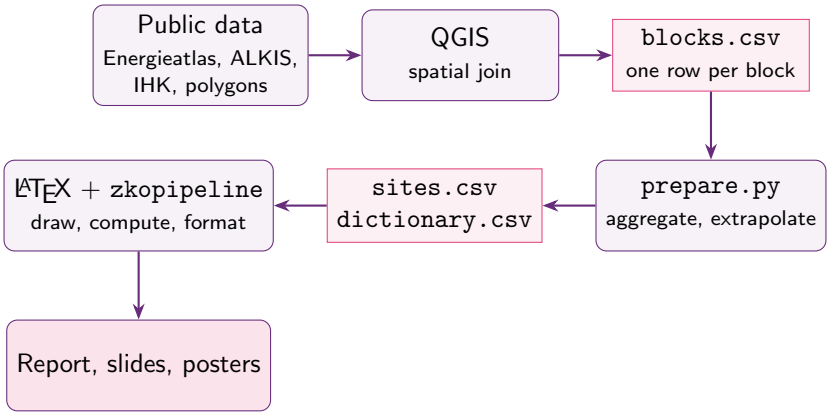


Figure 1: The pipeline. Boxes are processing steps, pink frames are files. In the first version, the step `prepare.py` was a spreadsheet.

could be derived from an earlier one.

2. **Machine format inside, local format outside.** All files use a decimal point and full precision. Rounding and the decimal comma (for German output) are applied only when a number is printed.

3 Stage 1 and 2: data and spatial join

3.1 Sources

The Berlin *Umweltatlas* and *Energieatlas* publish electricity, gas and district heating consumption for each statistical city block. Further layers came from the property register (ALKIS: buildings, listed monuments), the chamber of commerce (IHK: registered businesses with size classes) and the state statistical office. The boundaries of the sites were available as GeoJSON polygons.

Two properties of the block data shape everything that follows:

- A block can *have* consumption without a *published* value. Values are withheld when a block has very few consumers (data protection), or when a supplier did not deliver data.
- Withheld blocks are not a random sample. Few consumers per block often means one large consumer, so withheld blocks tend to consume *more* than average.

3.2 Selection rule: within or intersecting?

A site boundary rarely follows block boundaries. QGIS offers two natural selections: blocks lying *within* the polygon, and blocks that *intersect* it. Figure 2 shows the difference.

Within underestimates (it drops blocks that are partly on the site); *intersects* overestimates (it adds blocks that are mostly outside). The original study used both, compared the results, and redrew one boundary by hand where neither rule was acceptable. In this example, `prepare.py` uses both relations; the rule is a single constant (`INCLUDE`) so that its effect can be tested.

The QGIS export is a plain CSV with one row per block: the site it belongs to, the relation (`within` or `intersects`), area, number of buildings, and for each energy carrier a status (`reported`, `suppressed`, `none`) and the value where reported.

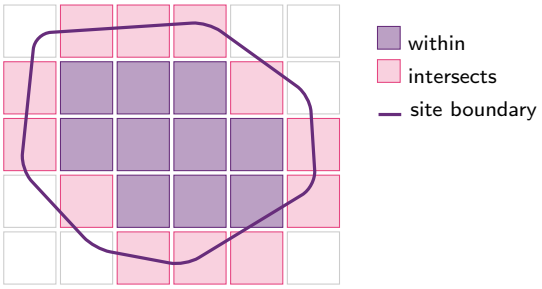


Figure 2: Schematic selection of blocks by a site boundary.

4 Stage 3: preparation

The script `data/prepare.py` replaces what was a spreadsheet in the first version. It has no dependencies beyond the Python standard library.

4.1 Extrapolating withheld values

For each site s and carrier c, let $B_{s,c}$ be the blocks with consumption and $R_{s,c} \subseteq B_{s,c}$ the blocks with a reported value e_b. The estimate assumes that withheld blocks consume as much as the average reported block:

$$\hat{E}_{s,c} \;=\; \frac{1}{|R_{s,c}|} \sum_{b \in R_{s,c}} e_b \;\cdot\; |B_{s,c}| \tag{1}$$

This estimator has two known weaknesses, both documented in the output:

- Because withheld blocks tend to be larger consumers, $\hat{E}$ is biased *downwards*.
- If no block of a site reports a value ($|R_{s,c}| = 0$), the estimate is zero although consumption exists. In the synthetic data this happens for district heating at *Old Airfield*: heat is used in 6 blocks, but none reports a value.

4.2 Emissions

CO_2 emissions use one emission factor per carrier, in tonnes per GWh of supplied energy. The values are those of the state energy balance used in the original project; they are constants at the top of the script.

Carrier	t CO_2/GWh
Electricity	365.22
Natural gas	202.77
District heating	219.65

4.3 Build-out factor

To estimate demand at full build-out, the current building density d_s (buildings per hectare) is compared with a target density $d^{*} = 10$, the density of the most built-up site in the original study:

$$g_s = \max\!\left(1,\; \frac{d^{*}}{d_s}\right), \qquad E_s^{\text{future}} = g_s \cdot E_s \tag{2}$$

The factor is an upper bound: it counts buildings, not floor area, and it assumes that new buildings consume like existing ones. The $\max(1, \cdot)$ is essential; without it, sites that are already denser than the target would *shrink* in the projection (see Section 8).

4.4 Output: data and dictionary

The script writes two files. `sites.csv` has one row per site and one column per quantity, sorted by total demand. `dictionary.csv` describes every column. Both are read by LaTeX; the table below is typeset directly from the dictionary.

5 The data dictionary

Column	Description	Unit	Source
`id`	Short identifier of the site		input
`name`	Name of the site		input
`blocks`	Blocks assigned to the site (within or intersecting)	count	QGIS
`area-ha`	Total area of assigned blocks	ha	QGIS
`buildings`	Number of buildings	count	QGIS (ALKIS)
`density`	Buildings per hectare: buildings / area-ha	1/ha	computed
`el-blocks`	Blocks with electricity consumption	count	QGIS (Energieatlas)
`el-reported`	Blocks with a reported electricity value	count	QGIS (Energieatlas)
`el-gwh-reported`	Sum of reported electricity consumption	GWh	computed
`el-gwh`	Extrapolated electricity consumption: mean per reported block times el-blocks	GWh	computed
`gas-blocks`	Blocks with natural gas consumption	count	QGIS (Energieatlas)
`gas-reported`	Blocks with a reported natural gas value	count	QGIS (Energieatlas)
`gas-gwh-reported`	Sum of reported natural gas consumption	GWh	computed
`gas-gwh`	Extrapolated natural gas consumption: mean per reported block times gas-blocks	GWh	computed
`heat-blocks`	Blocks with district heating consumption	count	QGIS (Energieatlas)
`heat-reported`	Blocks with a reported district heating value	count	QGIS (Energieatlas)
`heat-gwh-reported`	Sum of reported district heating consumption	GWh	computed
`heat-gwh`	Extrapolated district heating consumption: mean per reported block times heat-blocks	GWh	computed
`total-gwh`	Extrapolated total supply: el-gwh + gas-gwh + heat-gwh	GWh	computed
`co2-t`	CO2 from total supply, using emission factors per carrier	t	computed
`factor-raw`	Target density / density (may be below 1)		computed
`factor`	Build-out factor: max(1, factor-raw)		computed
`total-gwh-future`	Total supply at full build-out: total-gwh times factor	GWh	computed

In the first version the dictionary lived in two extra rows at the bottom of the data file (description and source), which the drawing macros skipped with a numeric test. Moving it to its own file keeps the data table rectangular and purely numeric, which every tool can read without special cases.

6 Stage 4: drawing with `zkopipeline`

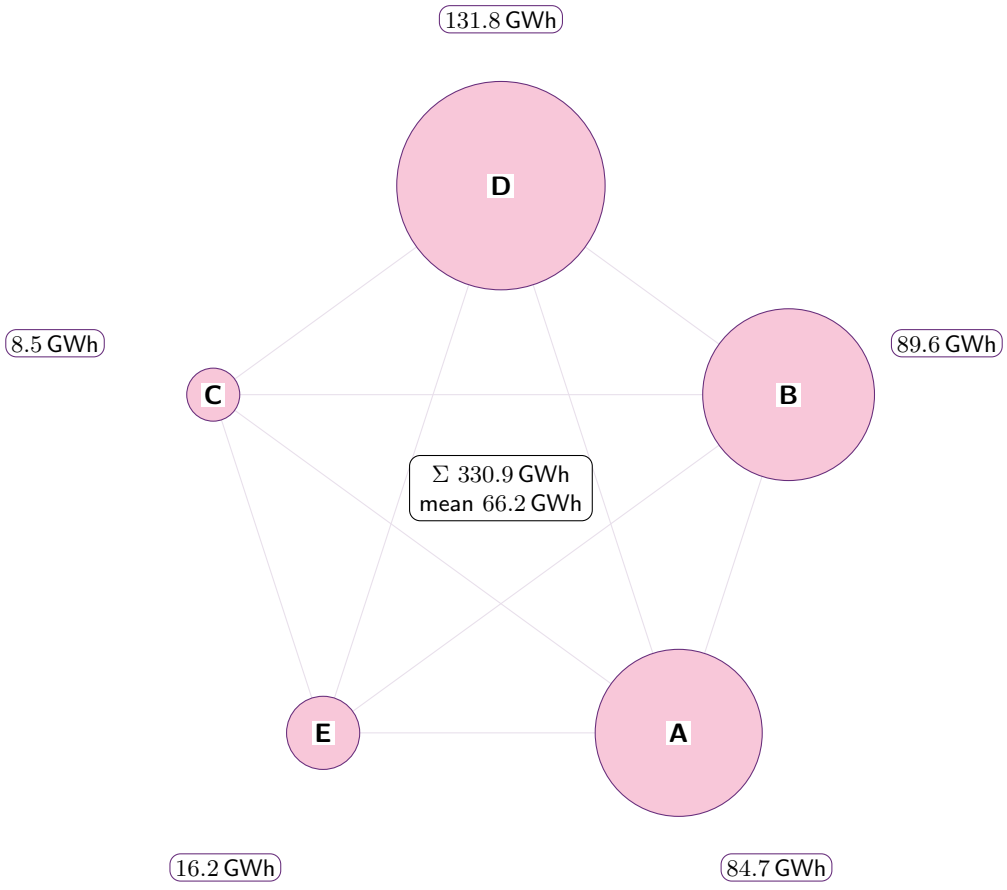
The package needs only the data loaded with `datatool`:

```
\usepackage{zkopipeline}
\DTLloaddb{sites}{data/sites.csv}
```

6.1 Circle map

`\zkomap` places the sites on a ring, connected by thin lines (the network motif of the original project). The *area* of each circle is proportional to the value, so the radius grows with its square root. The centre shows sum and mean.

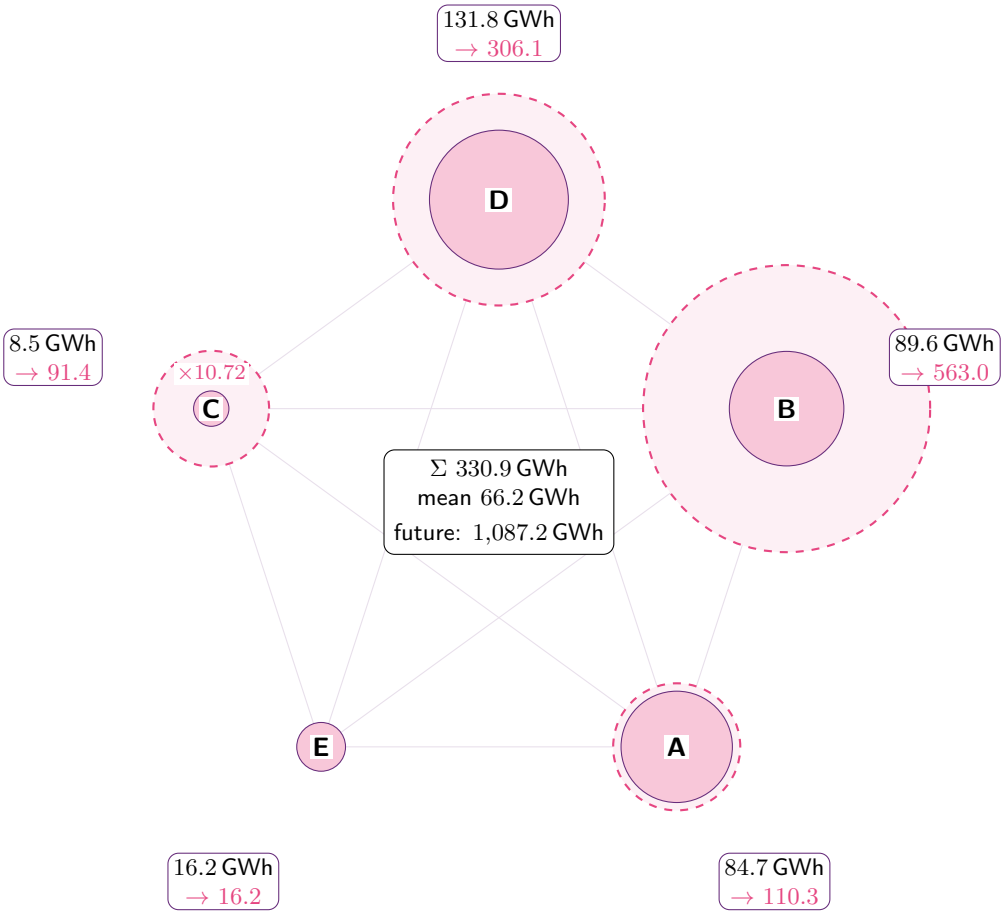
```
\zkomap[unit=GWh, scale=0.12]{sites}{total-gwh}
```



6.2 Projection overlay

With `future=<column>`, each value is multiplied by the factor in that column and drawn as a dashed circle. Sites whose factor is 1 get no overlay.

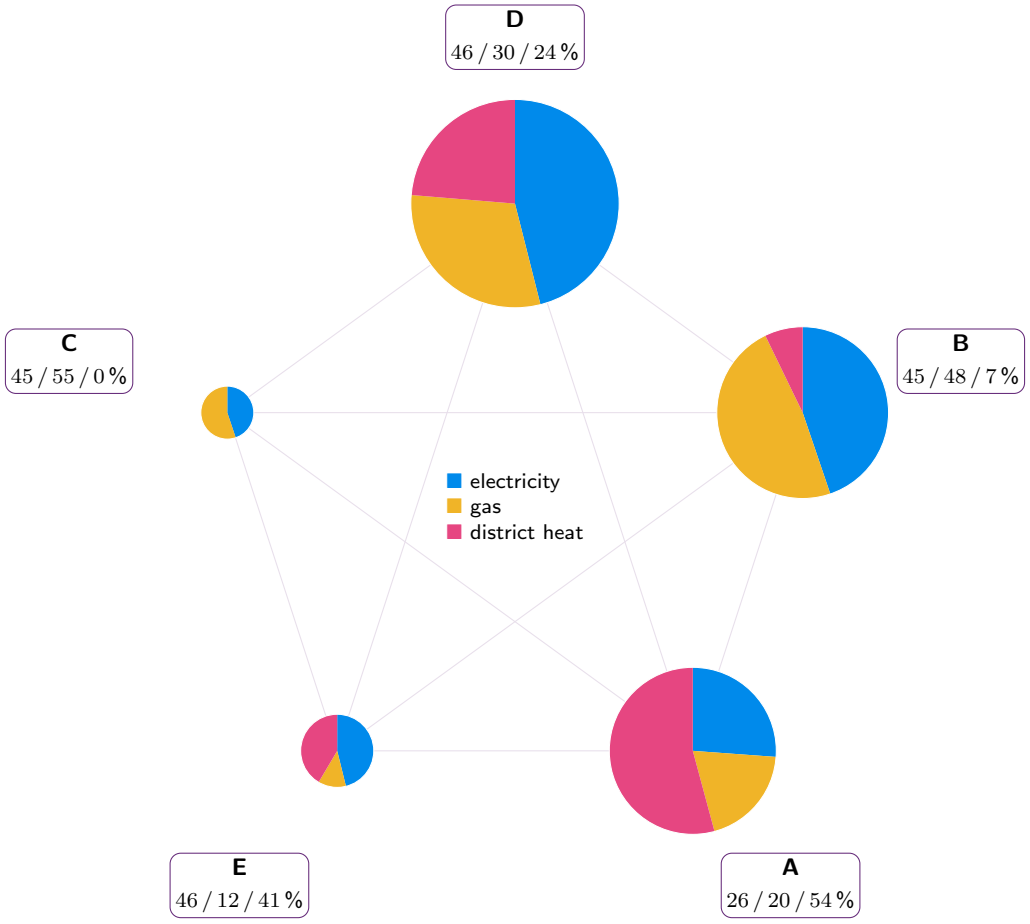
```
\zkomap[unit=GWh, scale=0.08, future=factor]{sites}{total-gwh}
```



6.3 Energy mix

`\zkopies` draws one pie per site with three shares, here electricity, gas and district heating. The pie area is proportional to the total. The label gives the three shares in per cent.

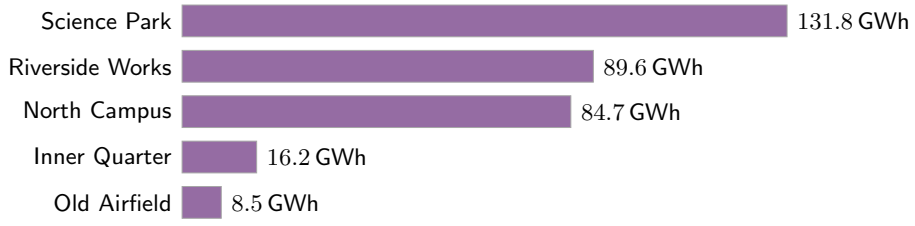
```
\zkopies[scale=0.12]{sites}{el-gwh}{gas-gwh}{heat-gwh}
```



6.4 Bar chart

Circle areas are good for showing orders of magnitude and poor for precise comparison. Where readers must compare values, `\zkobars` is the better choice. Bars appear in the order of the data, which `prepare.py` sorts by total demand. With `color=<column>`, each bar takes its colour from that column (a colour name per row), e.g. to match the colours of the same categories in a map.

```
\zkobars[unit=GWh, width=8]{sites}{total-gwh}
```



7 Numbers in the text

Every drawing macro leaves its results in global macros: `\zkoresult` holds the sum, `\zkofutureresult` the projected sum. The text then computes with these values instead of copying them.

7.1 An example paragraph

The source of the following paragraph contains no typed number:

> The five sites draw 330.9 GWh of energy per year, of which natural gas accounts for 32.1 %. Their supply causes about 90 kt of CO_2. At full build-out, demand could rise to 1,087.2 GWh, a change of +228.6 %, or 3.29 times today's level.

```
\zkototal{sites}{gas-gwh}{\gas}  \zkototal{sites}{co2-t}{\co}
The five sites draw \zkonum{\sitesnow}~GWh of energy per year, of which
natural gas accounts for \zkoshare{\gas}{\sitesnow}. Their supply causes
about \zkonum[0]{\co/1000}~kt of CO$_2$. At full build-out, demand
could rise to \zkonum{\sitesfuture}~GWh, a change of
\zkogrowth{\sitesnow}{\sitesfuture}, or \zkoratio{\sitesnow}{\sitesfuture} times ...
```

The values `\sitesnow` and `\sitesfuture` were taken from the projection map above, directly after drawing it: `\let\sitesnow\zkoresult \let\sitesfuture\zkofutureresult`. They must be saved at once, because the next chart overwrites `\zkoresult`.

7.2 Share, growth, ratio

Three macros, three different questions. Keeping them apart by name prevents the most expensive error of the first version.

Macro	Computes	Example: from 2,145 to 11,026
`\zkoshare{a}{b}`	$100 \cdot a/b$	514.0 % (share; wrong here)
`\zkogrowth{a}{b}`	$100 \cdot (b-a)/a$	+414.0 % (correct)
`\zkoratio{a}{b}`	b/a	5.14

Each macro takes an optional first argument to store the unrounded result, e.g. `\zkogrowth[\g]{a}{b}`, and prints n/a instead of dividing by zero.

7.3 Rounding and locale

Values are rounded only when printed, by `siunitx`. The same value can be shown in English or German notation by changing one setting:

Setting	Output of `\zkonum[2]{\sitesfuture}`
English (this document)	1,087.16
German	1.087,16

```
\sisetup{output-decimal-marker={,}, group-separator={.}}
```

8 Lessons learned

The first version produced a 121-slide deck, a report and eleven posters. A review of that version found the following errors. None was a mistake in the data; each was a weakness of the pipeline. The right column shows how this version rules it out.

Symptom	Cause	Remedy in this version
A 5.14-fold increase was reported as "growth of 514 %" instead of 414 %.	One macro computed a/b and was used both for shares and for growth.	Separate, explicitly named macros: `\zkoshare`, `\zkogrowth`, `\zkoratio`.
Three percentages in the text did not match the charts.	Numbers had been typed into the text from an earlier data version.	No typed numbers: every value in the text is read or computed from the data (Section 7).
Projected growth shown as +144 % instead of +137.9 %.	Values were rounded to 0.1 TWh *before* dividing.	Full precision in all files and computations; rounding only on output.
Nine dated copies of the data file.	Versioning by file name; old numbers survived in old copies.	One data file, generated by a script and versioned with Git.
Derived columns could not be reproduced.	The spreadsheet kept values only; the formulas survived as text descriptions.	`prepare.py` computes every derived column; the dictionary documents it.
Mixed decimal separators in the output; conversion code in every macro.	Data stored with decimal comma, but TikZ and `fp` need a point.	Decimal point in all files; `siunitx` formats the output.
Dense sites shown as "could grow by −3 %"; the projected total too low.	Build-out factors below 1 were applied and summed.	$g = \max(1, d^*/d)$ in `prepare.py` (Section 4).
Electricity use (GWh) and PV capacity (MWp) both labelled "GW".	One unit argument for a chart that combined two quantities.	Each column has its own unit in the dictionary; one quantity per chart.
"Division by zero" for a site without data.	The zero test compared strings: `0.00` is not `0`.	Numerical tests only: `\zkotest{a == 0}{...}{...}`.
"Dimension too large" with large values (e.g. tonnes of CO_2).	Comparisons via `\ifdim`, limited to about 16,383.	All arithmetic and comparisons with `\fpeval`.
Errors with some TEX installations only.	LATEX macros (site names) inside a data file, read before they were defined.	Data files contain plain text only.
Duplicate labels, broken cross-references ("Folie ??").	Frames copied to create variants, labels copied with them.	Labels derived from a single naming scheme; warnings treated as errors.

9 Building

```
python3 data/make_example_data.py   # synthetic blocks.csv (skip with real data)
python3 data/prepare.py             # sites.csv and dictionary.csv
pdflatex doku.tex                   # twice, for the table of contents
```

Requirements: Python 3.8 or later (standard library only) and a current TEX distribution with `tikz`, `datatool`, `siunitx`. Tested with TEX Live 2023 (`datatool` 2.32) and `pdflatex`.

To use real data, replace `blocks.csv` by the QGIS export (same columns) and adjust `SITE_NAMES` in `prepare.py`.

A Macro reference

Macro	Purpose
`\zkomap [opt]{db}{col}`	Circle map; stores `\zkoresult`, with `future` also `\zkofutureresult`
`\zkopies [opt]{db}{c1}{c2}{c3}`	One pie per site with three shares
`\zkobars [opt]{db}{col}`	Horizontal bars in data order
`\zkototal {db}{col}{\cs }`	Column sum, full precision, into `\cs`
`\zkonum [digits]{expr}`	Evaluate and print a number, rounded
`\zkoshare [\cs ]{a}{b}`	$100\,a/b$ in per cent
`\zkogrowth [\cs ]{a}{b}`	$100\,(b-a)/a$ in per cent, with sign
`\zkoratio [\cs ]{a}{b}`	b/a
`\zkotest {expr}{true}{false}`	Numerical condition
Options	`unit`, `scale` (cm per $\sqrt{\text{unit}}$), `ring` (radius), `future` (factor column), `width` (longest bar, cm), `mesh` (true/false), `color` (bars: column with a colour name per row)